

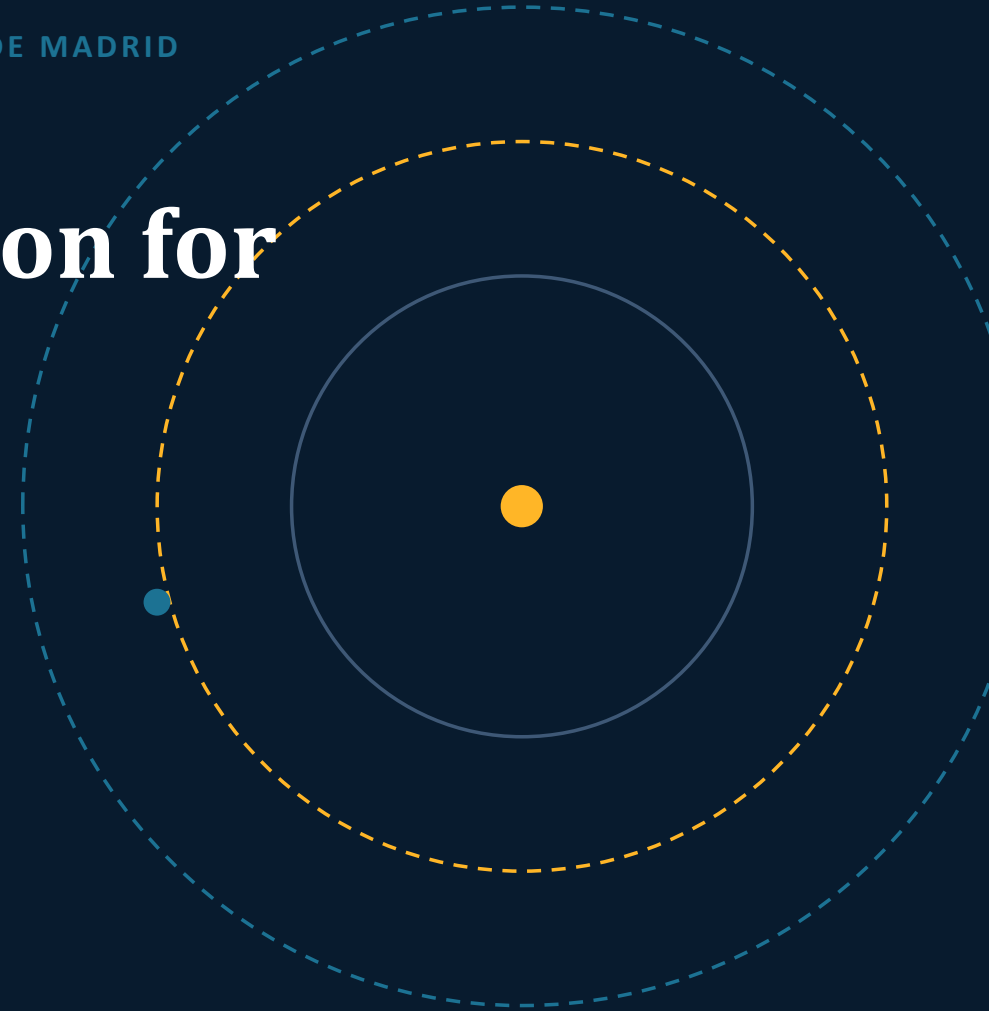
Adiabatic Quantum Optimization for Multi-Agent AI Workforce Orchestration

*QUBO Encoding, Hamiltonian Design, Spectral Analysis, and
Reproducible Quantum-Dynamics Evaluation*

Elijah Kim

Director: Alberto García García · Co-Director: Jesús García López de Lacalle

Master's Thesis Defense · July 19, 2026



AI teammates need a fair way to split the work

Today, many AI systems are not just one model answering one question. They are teams of AI “agents” that read shared notes, use tools, and pull tasks from a common to-do list.

Every time a new batch of tasks arrives, someone — or something — has to decide: which agent does which task, right now?



Agents

Different skills, different costs to run



Tasks

Different effort, different value



Shared backlog

Everyone pulls from one queue

This scheduling decision is a hard math problem

Deciding who does what isn't simple guesswork. It has real rules attached:



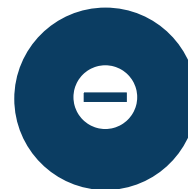
Capacity

Each agent can only hold so much work at once



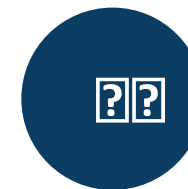
Affinity

Some tasks go faster together, on the same agent



Separation

Some tasks must never share an agent



Deferral

If nothing fits, a task can safely wait

With this many rules combined, the number of possible schedules explodes as more agents and tasks join. This is what mathematicians call an “NP-hard” problem — there is no shortcut formula, only smarter ways to search.

Four questions this thesis set out to answer

RQ1

Can we write down this scheduling problem as a math recipe (a QUBO) that always gives valid answers?

RQ2

How does the size of the “penalty” for breaking rules affect how well a quantum computer can search?

RQ3

Can smarter quantum techniques — a tuned schedule and a smarter driver — beat the plain, standard approach?

RQ4

What information would we need to actually run this on real quantum hardware one day?

What is a QUBO? Think of it as a scorecard

QUBO stands for “Quadratic Unconstrained Binary Optimization.”

That is a long name for a simple idea:

- Every decision is a switch: 0 (no) or 1 (yes) — e.g. “does Agent 1 get Task 3?”
- Good choices earn points. Bad choices (breaking a rule) lose points.
- The computer's job is simply: find the on/off pattern with the best total score.

The scorecard, in formula form

$$E(z) = c + \sum Q_{pp} z_p + \sum Q_{px} z_p z_x$$

Every task-to-agent switch is a z . Reward terms make good matches score higher; penalty terms make rule-breaking patterns score much worse — so the best-scoring pattern is automatically a valid schedule.

What is quantum annealing? A ball rolling to the lowest valley

A quantum computer can be steered so that its “ground state” — its most relaxed, lowest-energy state — matches the best answer to our scorecard.

1

Start simple

Begin in an easy state we fully understand — like a ball placed on a flat table.

2

Morph slowly

Gradually reshape the landscape into our scorecard's hills and valleys.

3

Land in the best answer

If the change is slow and smooth enough, the ball settles into the deepest valley — the best schedule.

Why it can go wrong

- Go too fast, and the system “jumps” past the best valley into a worse one.
- A narrow “gap” between the best valley and the next one makes jumping more likely — physicists call this an avoided crossing.
- **This thesis studies exactly how to avoid that trap for our scheduling problem.**

Turning the scheduling rules into precise math

Utility (u)	How good agent i is at task j
Capacity (C)	How much work agent i can hold
Affinity (a)	Bonus when related tasks share an agent
Deferral cost	Cost of leaving a task for later, instead of failing
Separation	Task pairs that may never share an agent

The goal, in plain words

Maximize total value — utility earned, plus affinity bonuses — minus the cost of any deferrals.

...while making sure every task is assigned or deferred exactly once, no agent is overloaded, and separated tasks never meet.

A key design choice: every task can always be safely “deferred.” This guarantees a valid schedule exists, even when things are overloaded.

Making sure the best-looking answer is always a legal one

If a rule gets broken, the scorecard must lose more points than any possible gain elsewhere — otherwise the computer might “cheat” and pick an invalid schedule because it scores well anyway.

The proof (Proposition 3.1)

If the penalty λ is bigger than B — the largest possible reward the schedule could ever gain — then every top-scoring answer is guaranteed to be a valid schedule.

This is a safe, conservative rule — it always works, on any instance.

But “safe” has a cost

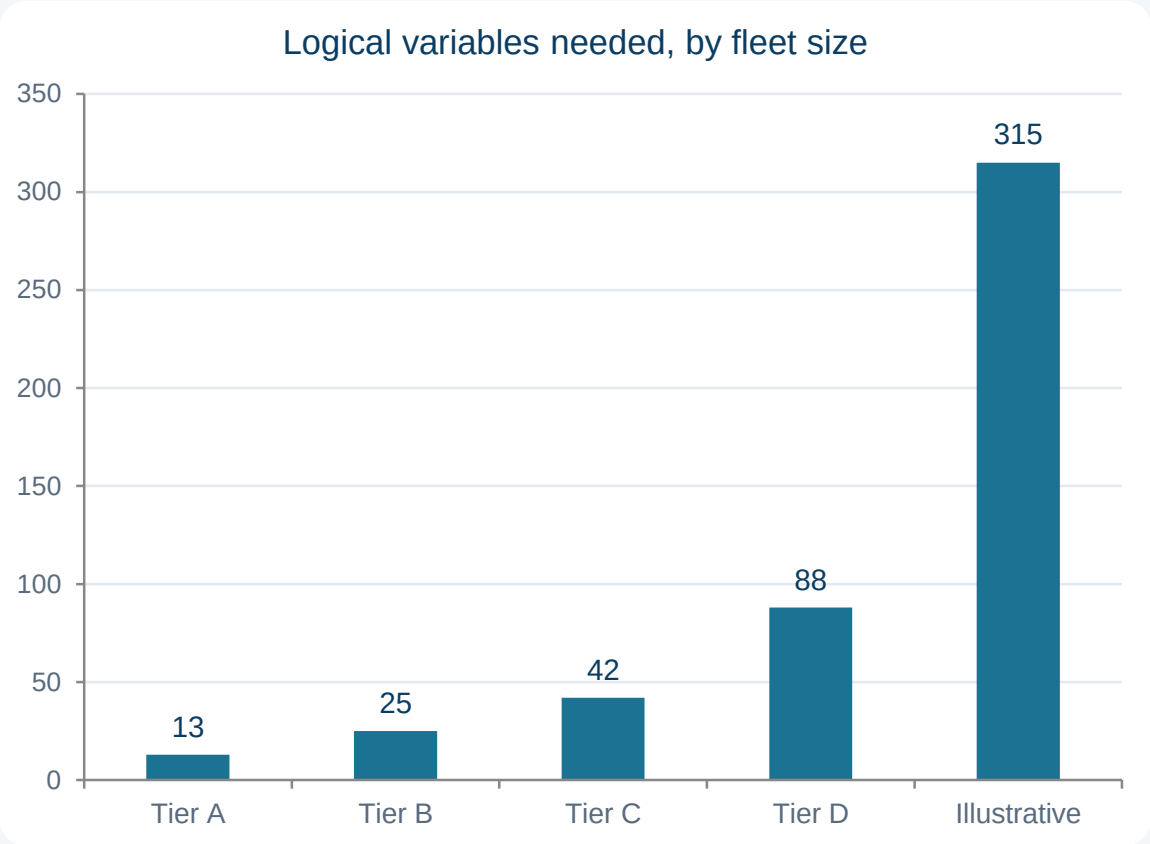
A very large, safe penalty squeezes all the useful score differences between valid schedules into a tiny sliver of the total range.

That makes it much harder for the quantum search to tell a great valid schedule apart from an okay one. Chapter 6 measures exactly how much this hurts — and how much smaller the penalty can safely be.

Every extra agent or task costs more qubits

The scorecard is translated into an “Ising” energy — the native language of quantum annealers — with no loss of information. But problem size grows fast:

Tier A	2×3 agents×tasks	13 variables
Tier B	3×4	25 variables
Tier C	4×6	42 variables
Tier D	6×10	88 variables
Illustrative	10×25	315 variables



Only the illustrative fleet's variable count is shown — no hardware embedding is claimed for it.

Three ways to help the quantum search find the answer

Baseline

Transverse-field driver

The standard, simplest way to explore all possibilities — our control condition.

Lever 1

Tune the schedule

Change how fast the system morphs, and where it spends extra time.

Lever 2

Tune the penalty

Use the smallest penalty that still guarantees valid answers.

Lever 3

Redesign the driver (XY)

Build task rules directly into the search, so it can't even explore invalid states.

How the experiments were built and checked

40

reproducible test cases across 4 fleet sizes

10

of them small enough for exact, full-detail quantum simulation

8,192

possible answers checked one by one for those 10 cases

What was tested for every case

- ✓ Classical solvers, for ground truth
- ✓ The plain baseline anneal at 3 different speeds
- ✓ A sweep of 7 penalty sizes
- ✓ 3 different schedules
- ✓ The rule-preserving XY driver
- ✓ Spectral “gap” measurements on 3 representative cases

The classical baseline: exact, and very fast

Before touching quantum dynamics, the thesis confirms ordinary computers can already solve these problems well — this sets the bar every quantum result is judged against.

< 58 ms

Worst-case time for an exact solver to find the perfect schedule

100%

Match between the QUBO's best answer and the exact solver, every time

74–98%

Score reached by a simple “greedy” rule-of-thumb, depending on fleet size

Exact MILP remains the practical choice for production use at these sizes — the interesting quantum question is different: why does a formally correct QUBO still challenge a quantum solver?

The plain, standard quantum anneal struggles

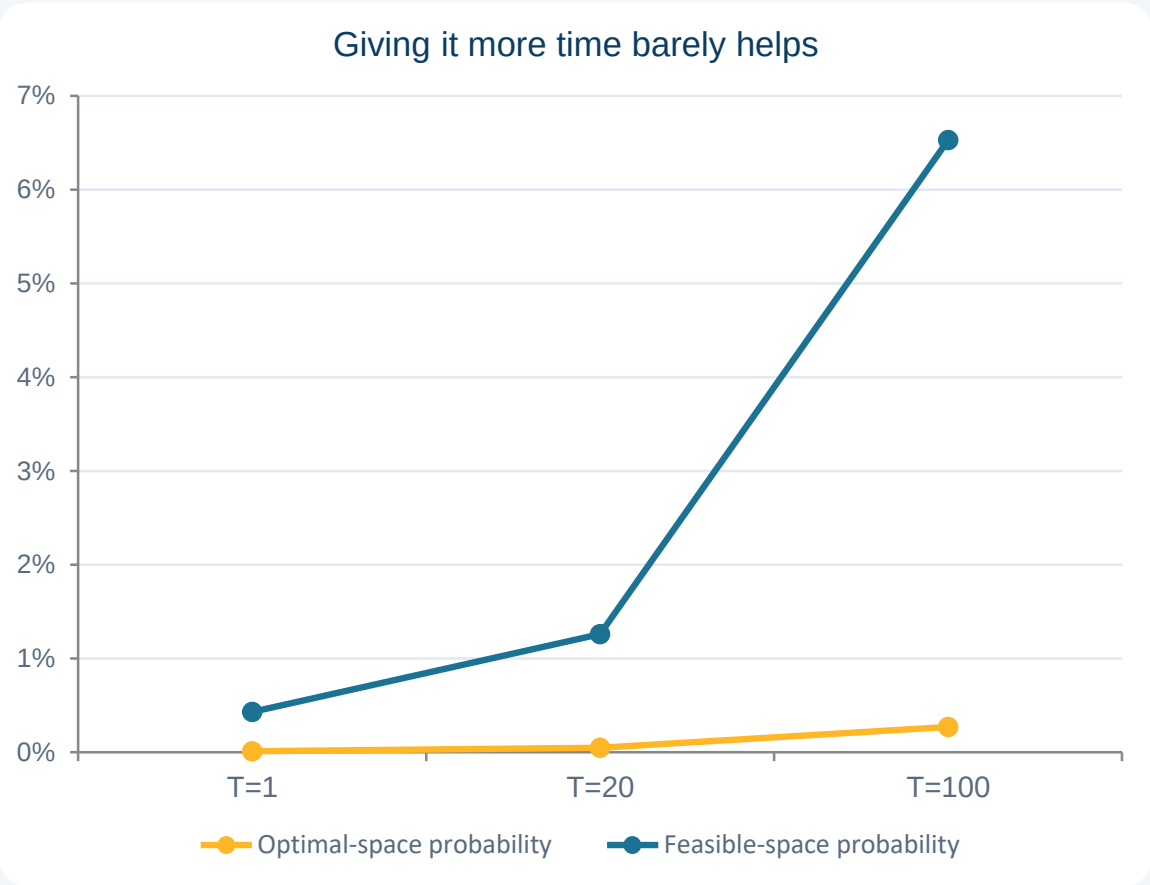
Running the textbook approach — the transverse-field driver on a straight-line schedule — is treated as the control condition. Even given plenty of time, it rarely lands on the best answer.

0.27%

chance of landing on the best possible schedule

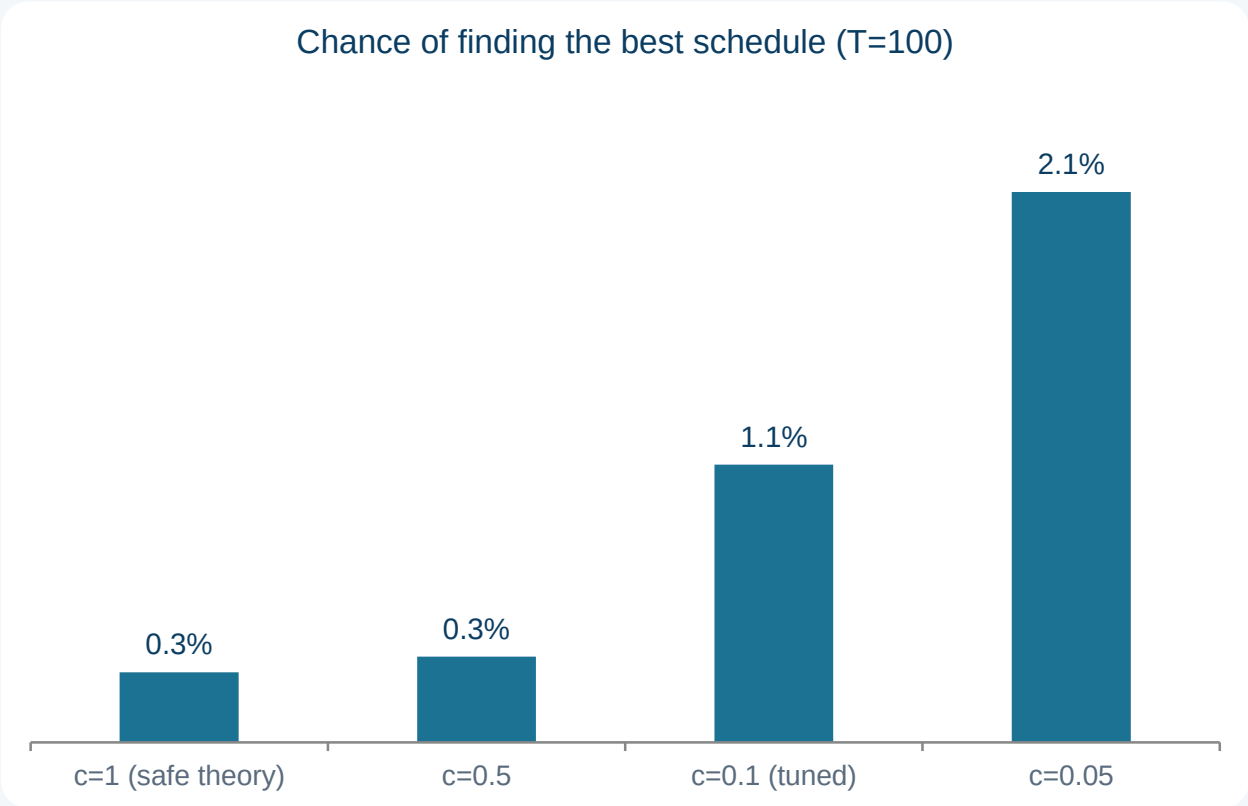
6.5%

chance of landing on any valid schedule at all



A smaller, carefully-checked penalty helps a lot

Shrinking the penalty to one tenth of the safe theoretical value — while still checking by hand that every answer stays valid — frees up room for the search to tell good schedules apart.



4×

higher chance of the best answer, moving from the safe setting to the tuned one

Caution: this tuned value (c=0.1) is only certified because all 8,192 answers on these 10 small cases were checked by hand. It does not yet scale to bigger fleets on its own.

Smooth isn't automatically better — targeted is

✗ A gentler start/end schedule

Rounding off the schedule's start and end (“endpoint smoothing”) did NOT help. At T=100 it slightly lowered both the chance of the best answer and of any valid answer.

It doesn't fix the real bottleneck, which sits deep inside the schedule, not at the edges.

✓ A schedule aimed at the bottleneck

Spending extra time exactly where the search is most likely to get stuck (found by measuring the “gap”) DID help — but only once given enough total time.

3.7×

higher chance of
the best answer

3.3×

higher chance of
any valid answer

(measured on 3 representative cases, at T=100)

Redesigning the driver gives the biggest win by far

Instead of penalizing rule-breaks, the XY driver makes it physically impossible for the search to ever leave the “one valid choice per task” zone. This shrinks the space the search has to explore from 8,192 states down to just 432.

1.82%

chance of the best schedule
(vs. 0.27% baseline)

46.9%

chance of any valid schedule
(vs. 6.5% baseline)

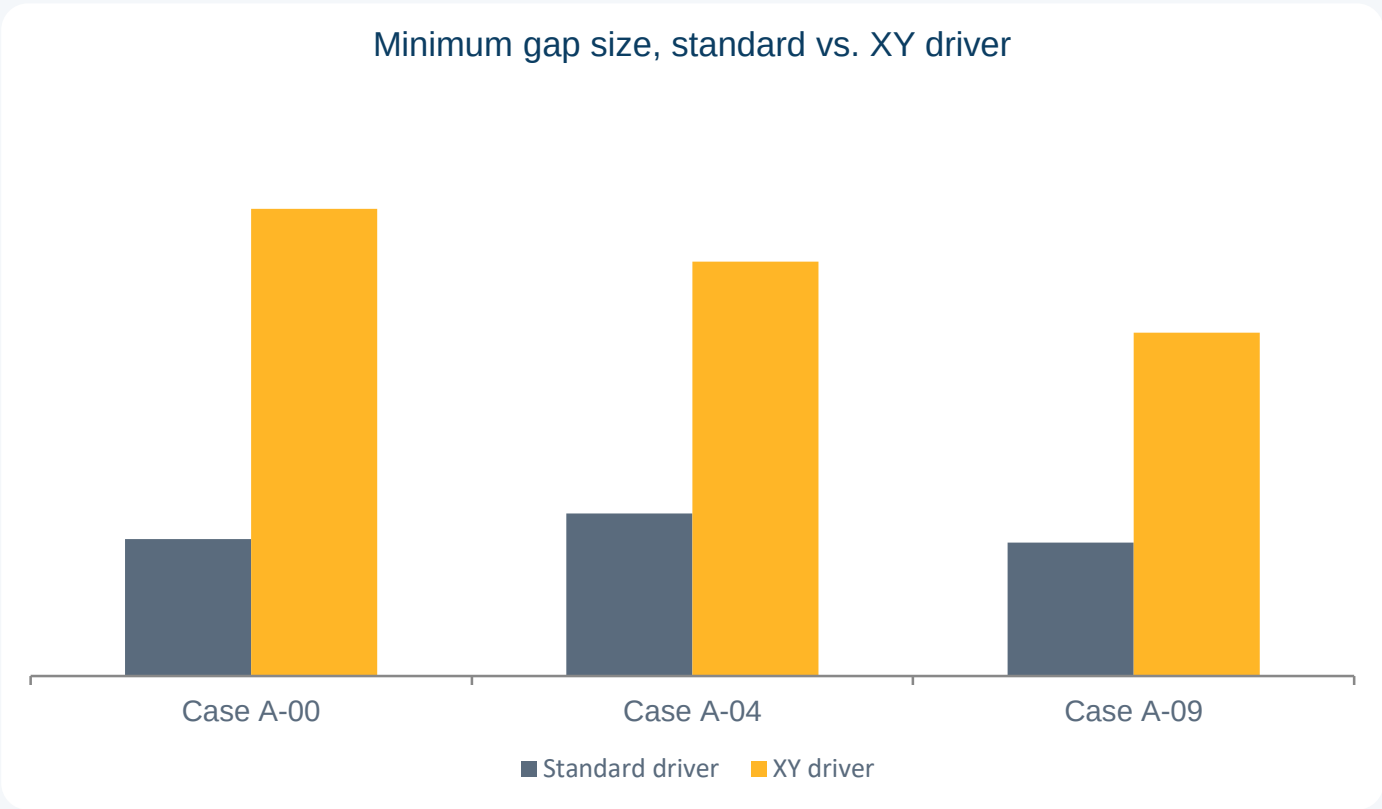
80.8%

chance of hitting the best
answer within 100 tries

That is roughly 6.8× the chance of the best answer, and 7.2× the chance of any valid answer, compared to the plain baseline.

Why it works: the XY driver widens the danger zone

Remember the “narrow gap” risk from earlier — the point where the search is most likely to jump to the wrong answer. Measuring that gap directly shows the XY driver makes it noticeably wider.



2.5–3.4×

wider gap with the
XY driver, across all
three test cases

A wider gap means the search can safely move a little faster through the riskiest moment without jumping to the wrong answer — though it is still a narrow gap overall, not a trivial problem.

Each fix works through a different mechanism

1

Smaller penalty

Mostly ranks good valid schedules better against each other

2

Targeted schedule

Spends time exactly where the search is most fragile

3

XY driver

Removes the invalid zone from the search entirely — biggest, broadest gain

Takeaway: *these are not competing fixes — they attack different parts of the same bottleneck, and could be combined in future work.*

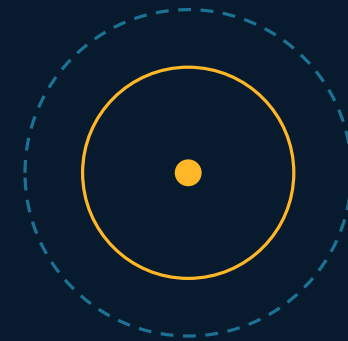
What this is NOT, and the real hardware plan

This thesis does NOT claim:

- ✗ A quantum speed advantage over classical solvers
- ✗ Real hardware results — all runs are noiseless simulations
- ✗ That the tuned schedule works for problems it wasn't built from
- ✗ That MILP should be replaced in production today

The path to real hardware (D-Wave)

- 1 Run the 10+10 smallest cases on a named solver, with full settings recorded
- 2 Sweep embeddings, chain strengths, and anneal times
- 3 Try both forward and reverse annealing
- 4 Report raw results before any clean-up, at least 1,000 tries per setting
- 5 Compare honestly against the classical baseline



A reproducible, honest map of why the naive anneal fails — and what actually fixes it

- A correct, provably-safe QUBO for real multi-agent scheduling rules
- Clear evidence that penalty scale and driver design — not just time — drive quantum performance
- The XY driver: the strongest result, 6.8× better odds of the best schedule
- A full, versioned bundle: 40 instances, code, raw results, and a ready-to-run hardware protocol

Thank you. Questions welcome.